memcpy 为什么不能拷自己：restrict 的秘密

嵌入式 Linux 手记　2026-10-20　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。这段代码想把前 4 个字节前移两格，输出是什么？

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char msg[] = "1234567890";
    memcpy(msg + 2, msg, 4);   /* "1234" 搬到 msg[2] 起的位置 */
    puts(msg);
    return 0;
}
```

两个候选答案都言之有理：`"1212347890"`，老老实实先读后写；`"1212127890"`，边写边读把数据踩脏了。两个都见过的人不在少数，因为标准给的答案是：输出是什么，根本不构成一个问题。源与目的重叠时 `memcpy` 的行为未定义（C11 §7.24.2.1 明文条款），任何结果都"合法"。篇 11 结尾说过，这一篇进标准库的发动机舱：`memcpy` 与 `memmove` 的一字之差里，藏着 C 标准发给优化器的许可证。

一、restrict：一张签了字的承诺书

两个函数的原型摆在一起，差异全在限定符上：

```c
void *memcpy(void *restrict s1, const void *restrict s2, size_t n);
void *memmove(void *s1, const void *s2, size_t n);      /* 一个 restrict 都没有 */
```

`restrict` 是 C99 引入的关键字（C++ 至今没有，编译器用 `__restrict` 扩展补位），语义写在 C11 §6.7.3.1：凡通过 restrict 指针访问的对象，在指针存续期内不得再经其他途径修改，除非那个"其他途径"也是从这条 restrict 指针派生出来的。翻译成人话：你向编译器承诺，这块内存只有这一条道。

承诺换回什么？编译器从此不必再假设"两个指针可能偷偷指向同一处"。看看这个循环：

```
void add(int *a, int *b, int *c, int n) {
    for (int i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}
```

`c` 万一与 `a` 指向同一块内存，写 `c[i]` 就改了 `a[i]`，下个迭代读到的将是新值。编译器证明不了不会重叠，只能保守：按最坏情况老老实实按序读写。给三个形参都加上 `restrict`，"重叠不发生"就成了既定事实，循环立刻可以向量化、按块重排、合并访存。篇 8 讲过 UB 的折叠威力，`restrict` 是它的温和版本：不是"发生了就天下大乱"，而是"你保证不发生，我按不发生来优化"。`memcpy` 两个参数都签了这份承诺书，于是源与目的重叠的那一刻，契约崩塌，行为逐出讨论范围。

再看一眼原型里的 `const void * restrict s2`：`const` 承诺"我不改你"，`restrict` 承诺"我走的是唯一一条道"，两个限定符各管一摊、互不替代。篇 7 拆过标准库签名的契约语言，`strcpy` 用 `const` 分工读写两侧；`memcpy` 更进一步，把"不许重叠"也写进了签名。

二、重叠时到底发生什么：一个字节污染剧本

UB 落到机器上是什么样？拿逐字节正向搬运当模型，这也是最朴素的实现思路，`src[i]` 读一个写一个：

```
msg:  1 2 3 4 5 6 7 8 9 0
      拷 msg[0]→msg[2]:   1 2 1 4 5 6 7 8 9 0
      拷 msg[1]→msg[3]:   1 2 1 2 5 6 7 8 9 0
      拷 msg[2]→msg[4]:   读到的是刚写入的 '1'! → 1 2 1 2 1 6 7 8 9 0
      拷 msg[3]→msg[5]:   读到的是刚写入的 '2'! → 1 2 1 2 1 2 7 8 9 0
```

`src[2]` 原本是 `'3'`，轮到读它时已经被自己前面两步覆盖成了 `'1'`。数据被自己的写入踩脏，这正是第二个候选答案 `"1212127890"` 的来历。

真实机器比模型更捉摸不定。小拷贝常被编译器内联成一两条 mov 指令，读在写前，结果恰好"对"；调库走 glibc 的宽块实现，一条宽寄存器指令搬运 32 甚至 64 字节，先整块读后整块写的路径结果也对，按块循环的路径又会踩脏。同一个源文件，换优化级别、换编译器、换一台 CPU，两种输出都见过。这恰是 UB 的实践含义：不是"一定出错"，是"没有任何一种结果你能指望"。"看起来对"最危险——它把雷埋到了上线之后。



图：逐字节正向搬运的字节污染剧本。拷到第三步时，`src[2]` 已被前面两步覆盖，读到的不再是原来的 `'3'`。

三、memmove 的存在理由：一张不带承诺的许可证

重叠前移是正经需求：环形缓冲区取数据、文本编辑器删字符、消息队列搬包，全是 `src` 在后 `dst` 在前的场景。所以标准留着 `memmove`：不带 restrict，也就没有契约可违反；标准对它的要求写在 §7.24.2.2，像临时数组先拷再写一样正确（as-if）。

实际实现不用真开临时区。重叠只有两种几何关系，判定方向就能安全通过：

```
char msg[] = "1234567890";
memmove(msg + 2, msg, 4);   /* dst 在 src 之后：从后往前搬 */
puts(msg);                  /* 稳定输出 "1212347890" */
```

从尾部倒着搬，`src` 的每个字节都在被覆盖之前离开，污染无从发生；`dst` 在 `src` 之前就正向搬。一个方向判断，换来来者不拒。代价呢？在 glibc 的 x86-64 移植里几乎为零——`memcpy` 与 `memmove` 共享同一套实现（`memcpy` 的汇编入口直接进 `memmove` 的实现族），`nm /lib/x86_64-linux-gnu/libc.so.6` 亲眼可见两个符号指向同处。标准契约依旧一式两份：可移

植代码不能指望所有平台都这样合体，但"不确定重叠与否就无脑用 memmove"在主流平台上不亏。

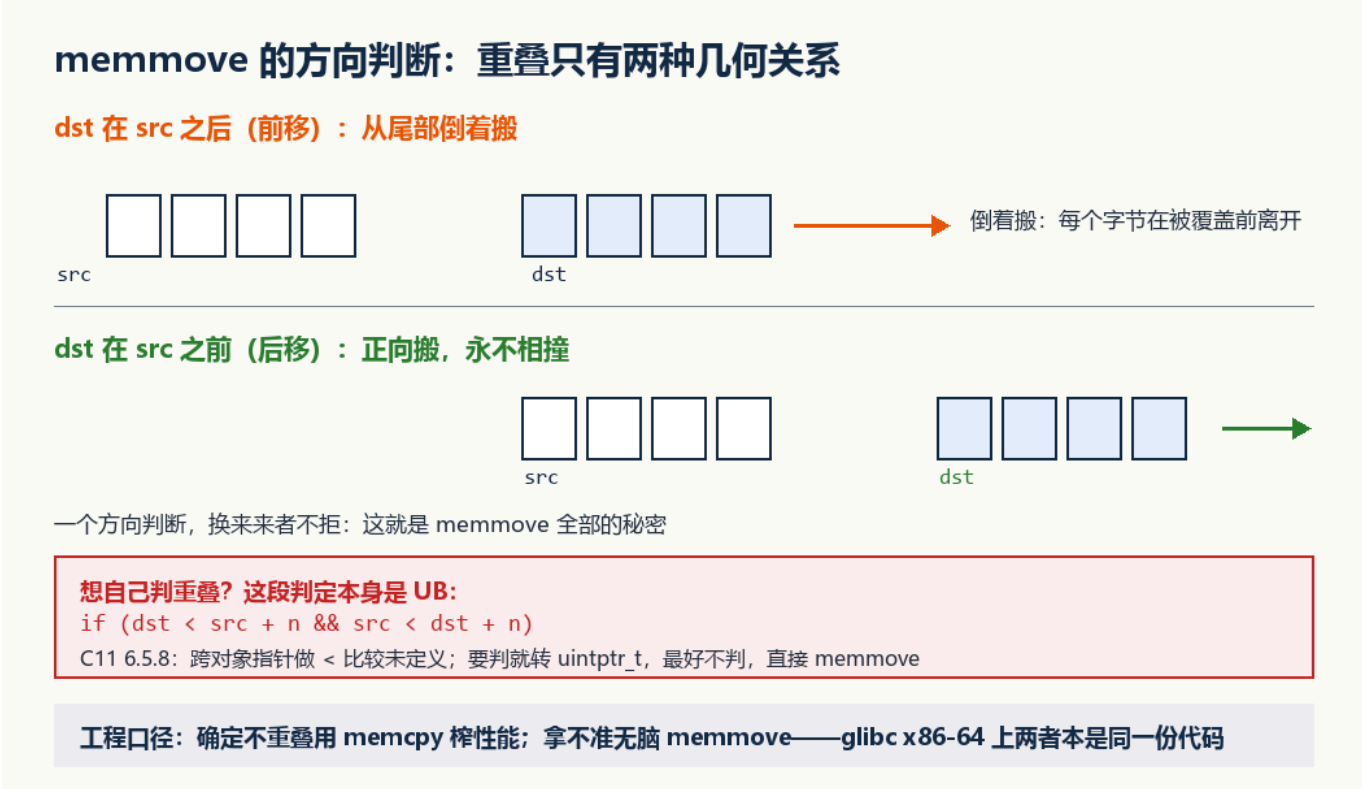
四、想自己判重叠？门外还埋着一个 UB

有读者会想到精打细算：先判重叠，不重叠走 `memcpy`，重叠走 `memmove`，两头收益都拿到。直觉写法长这样：

```
if (dst < src + n && src < dst + n)      /* 区间相交判定…… */
    memmove(dst, src, n);
else
    memcpy(dst, src, n);
```

这行判定本身是 UB。C11 §6.5.8 规定：用 `<` 这类关系运算符比较指针，仅在两个指针指向同一聚合对象（或其尾后一位置）时有定义；指向两个不相干内存块的指针做 `<` 比较，行为未定义。这不是学究条款：分段内存架构上指针比较不是简单的数值大小，优化器也可能基于"关系比较蕴含同对象"做推导。跨对象唯一保底的是 `==` 与 `!=`（判相等永远合法）。

正路两条：把指针转成 `uintptr_t`（`<stdint.h>`，C11 §7.20.1.4）再做整数比较，转换本身是实现定义，但整数比较没有歧义；或者干脆不判：重叠概率低的调用点直接 `memcpy` 并注释清楚，拿不准就 `memmove`。为省一个方向判断在每次调用前先做一次 UB 比较，账算下来是负的。



图：memmove 的方向判断。重叠只有两种几何关系，各选一个搬运方向即可安全通过；自己判重叠的 `<` 比较反而是 UB。

写在最后

1. memcpy 两个 `restrict` 是写给优化器的承诺书（C11 §6.7.3.1）；源与目的重叠即违反契约，C11 §7.24.2.1 明文 UB。
2. 重叠的典型恶果是数据被自己的写入踩脏；真实机器上结果随内联、库实现、CPU 漂移，"看起来对"不是承诺。
3. memmove 不带 restrict，标准按"先拷临时区"的 as-if 效果要求它；实现靠方向判断省掉临时区，重叠前移从此合法。
4. 自判重叠的 `dst < src + n` 本身是 UB（§6.5.8 关系比较仅限同一对象内）；要判就转 `uintptr_t`，最好不判。
5. 工程口径一句话：确定不重叠、要榨性能，用 memcpy；拿不准，无脑 memmove，glibc x86-64 上两者本是同一份代码。

发动机舱还有一台更巧妙的原动机：`memcpy` 靠 `void*` 抹平一切类型，`qsort` 靠它实现 C 版泛型。下一篇《C 没有 class 怎么写泛型：void* 的真实用法》，把 `qsort` 的内部拆开看。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。